

Module 4: SNMP, ASN.1, Physical Layer



Module 4: SNMP, ASN.1, Physical Layer

- SNMP
- ASN.1
- Physical Layer:
 - Data and signals
 - Digital transmission
 - Analog transmission
 - Bandwidth utilization
 - Transmission media

9.1 Network Management – Introduction

Network Management = monitoring, testing, configuring and troubleshooting network components to satisfy organizational requirements.

Goal:

- Smooth and efficient network operation
- Required Quality of Service (QoS)
- Proper operation of network components

A Network Management System uses:

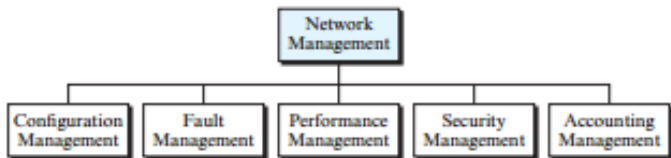
Hardware + Software + Humans

Network Management – Introduction

ISO defines five major areas:

Area	Main Purpose
Configuration	Manage network components and their configuration
Fault	Detect, isolate, correct and record failures
Performance	Monitor and control network efficiency
Security	Control access according to policy
Accounting	Control resource usage through charges

Figure 9.1 *Areas of network management*



Configuration and Fault Management

1. Configuration Management

The system must know the current status of each network entity and its relationship with other entities.

A. Reconfiguration

- **Hardware:** replace computer, move router, add/remove subnetwork – mainly manual.
- **Software:** install/update software or OS – mostly automatable.
- **User accounts:** add/delete users and manage individual/group privileges – partly automatable.

B. Documentation

- **Hardware:** maps and specifications (type, serial number, vendor, purchase, warranty).
- **Software:** type, version, installation time, license.
- **Users:** accounts and access privileges.

Configuration and Fault Management

2. Fault Management

Reactive Fault Management

- ① **Detect** – locate abnormal condition.
- ② **Isolate** – limit impact and notify users.
- ③ **Correct** – repair/replace faulty component.
- ④ **Record** – location, cause, action, cost and time.

Why record faults?

- Helps solve recurring problems.
- Frequent failures may indicate a major system problem.
- Statistics support performance management.

Proactive Fault Management

- Attempts to prevent faults, Replace components before expected lifetime, Reconfigure locations where faults occur frequently.

3. Performance Management

Ensures the network operates as efficiently as possible.

Measured using:

- **Capacity** – maximum network load.
- **Traffic** – packets/bytes travelling inside and outside the network.
- **Throughput** – data handled by a device/network.
- **Response Time** – time from user request to service delivery.

Excessive traffic or capacity usage can cause reduced data rate, blocking and increased response time.

SNMP can be used for performance management.

4. Security Management

Controls access to the network according to a predefined policy.

Encryption

- Provides privacy for users.

Authentication

- Forces users to identify themselves.

Policy → Authentication → Authorized Access

5. Accounting Management

5. Accounting Management

Controls users' access to network resources through charges.

Charges may be assigned to:

- Individual users
- Departments
- Divisions
- Projects

Charging may mean budgeting/debiting, not necessarily cash transfer.

Reasons:

- Prevent resource monopolization.
- Prevent inefficient resource use.
- Support short- and long-term planning.

**Network Management = Configuration + Fault + Performance +
Security + Accounting**

9.2 Simple Network Management Protocol (SNMP)

Network Management



9.2 SNMP – Introduction

Simple Network Management Protocol (SNMP)

- SNMP is a framework for managing devices in an Internet using the TCP/IP protocol suite.
- It provides fundamental operations for monitoring and maintaining an Internet.
- SNMP is an **application-level protocol**.
- It is designed to manage devices made by different manufacturers.
- It can operate over different physical networks.

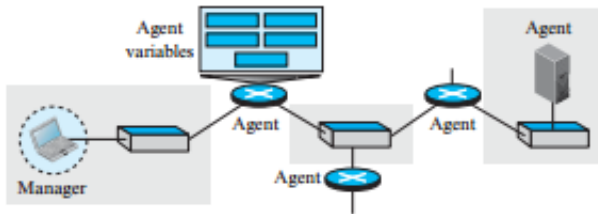
Key Idea

SNMP separates network management from the physical characteristics of managed devices and the underlying networking technology.

Typical managed devices:

Routers	Servers	Hosts	Other Network Devices
---------	---------	-------	-----------------------

Figure 9.2 *SNMP concept*



Managers and Agents

Manager

- A management station.
- Usually a host running the SNMP client program.
- Controls and monitors a set of agents.
- Requests information from agents.
- Can change values in the agent database.
- Can remotely force an agent to perform an action.

Agent

- A managed station.
- Usually a router or host running the SNMP server.
- Maintains performance information in a database.
- Provides requested information.
- Monitors its environment.
- Can send a warning called a **Trap**.

Three Basic SNMP Management Operations

1. Manager checks an agent

- Manager requests information reflecting the behaviour of the agent.
- Example: number of packets received and forwarded.

2. Manager forces an agent to perform a task

- Manager changes a value in the agent database.
- Example: resetting a reboot counter can force a remote reboot.

3. Agent contributes to management

- Agent continuously monitors its environment.
- If it detects an unusual situation, it sends a **Trap** to the manager.

Operation	Direction
Request information	Manager → Agent
Force action	Manager → Agent
Warning / Trap	Agent → Manager

Manager Request / Set / Agent Trap

9.2.2 Management Components

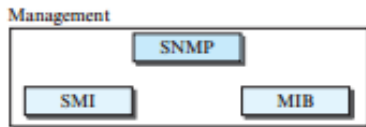
Internet network management uses three cooperating components:

Component	Role
SNMP	Defines packet/message format and reads or changes object values.
SMI	Defines general rules for naming objects, defining data types and encoding objects and values.
MIB	Defines the collection of managed objects, their names, types and relationships for an entity.

SNMP + SMI + MIB

- **SMI** provides the rules.
- **MIB** defines the managed objects.
- **SNMP** performs management operations on those objects.

Figure 9.3 *Components of network management on the Internet*



Understanding SNMP Through a Programming Analogy

The comparison of SNMP management with writing a program.

Programming	SNMP	Purpose
Language syntax	SMI	Defines rules
Variable declaration	MIB	Defines objects and types
Program statements	SNMP	Reads/changes values

SMI – Syntax

- Naming rules
- Data types
- Range and length
- Encoding rules

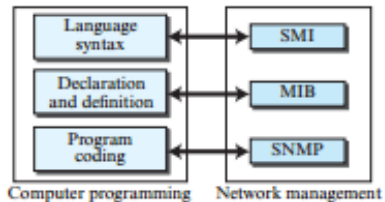
MIB – Object Declaration

- Names each object.
- Defines the type and relationship

SNMP – Program Coding

- Stores values.
- Changes values.
- Retrieves values.
- Interprets values.

Figure 9.4 *Comparing computer programming and network management*



9.2.3 SNMP Overview – A Simple Scenario

Suppose a manager wants to know:

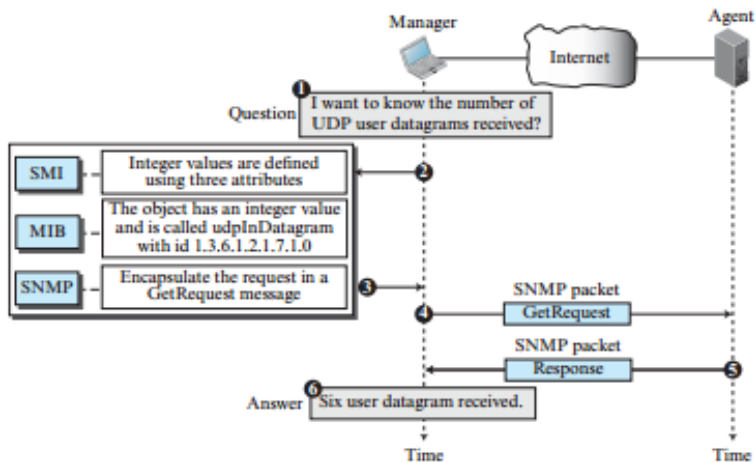
How many UDP user datagrams has an agent received?

The three components cooperate:

- 1 **MIB** identifies the object containing the required information.
- 2 **SMI** provides rules for identifying and encoding the object's name.
- 3 **SNMP** creates a **GetRequest** message.
- 4 The message is sent from manager to agent.
- 5 Agent returns the requested value in a response.

MIB → SMI → SNMP GetRequest → Agent → Response

Figure 9.5 Management overview



9.2.4 SMI – Name, Type and Encoding

Structure of Management Information version 2 (SMIv2)

SMI emphasizes three attributes of every managed object:

Attribute	Purpose
Name	Gives the object a unique identity.
Data Type	Defines what kind of data can be stored.
Encoding	Defines how data is represented for transmission.

SMI functions

- Name objects.
- Define data types.
- Show how data is encoded for transmission.

SMI: Name + Type + Encoding

SMI Object Identifier (OID)

Every managed object must have a **unique name**.
SMI uses a hierarchical **Object Identifier (OID)** tree.

iso.org.dod.internet.mgmt.mib-2



1.3.6.1.2.1

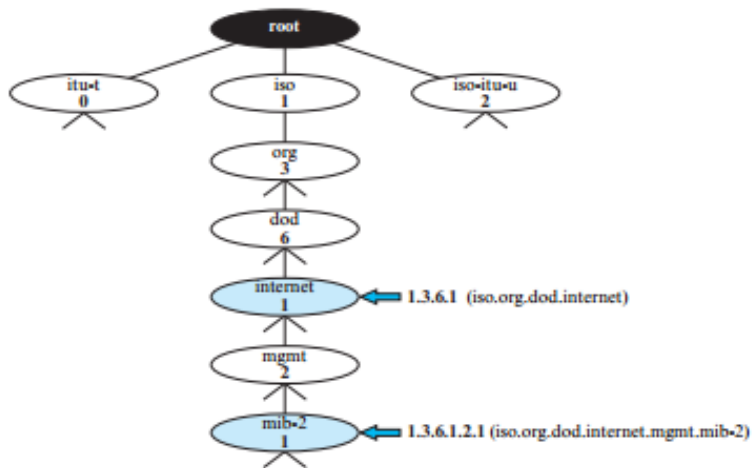
Two representations:

- **Name-dot notation:** used by people.
- **Integer-dot notation:** used by SNMP.

Objects used in SNMP are located below mib-2.
Therefore their identifiers begin with:

1.3.6.1.2.1

Figure 9.6 Object identifier in SMI



SMI Data Types

SMI uses ASN.1 definitions and adds additional definitions.

Two broad categories:

Simple Types

Atomic data types.

- INTEGER
- Integer32
- Unsigned32
- OCTET STRING
- OBJECT IDENTIFIER
- IPAddress
- Counter32
- Counter64
- Gauge32
- TimeTicks
- BITS
- Opaque

Structured Types

Sequence

Combination of simple data types, not necessarily of the same type.

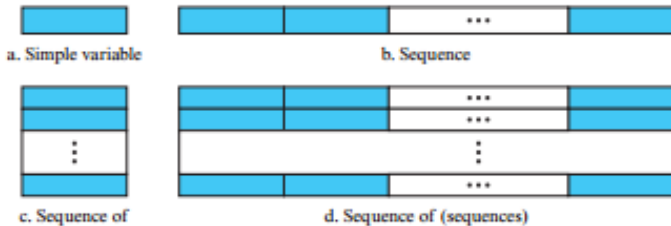
Similar to C struct/record

Sequence of

Collection of data types of the same type.

Similar to C array

Figure 9.7 Conceptual data types



SMI Encoding – Basic Encoding Rules (BER)

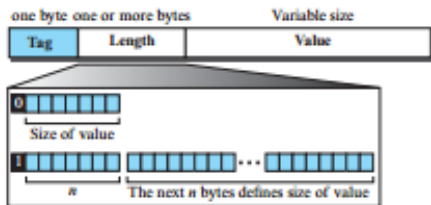
SMI uses **Basic Encoding Rules (BER)** to encode data transmitted over the network.

Tag		Length		Value
-----	--	--------	--	-------

- **Tag:** identifies the data type.
- **Length:** specifies the size of the value.
- **Value:** contains the encoded data.

Data Type	Tag	Data Type	Tag
INTEGER	02	IPAddress	40
OCTET STRING	04	Counter	41
OBJECT IDENTIFIER	06	Gauge	42
NULL	05	TimeTicks	43
SEQUENCE / SEQUENCE OF	30	Opaque	44

Figure 9.8 Encoding format



BER Encoding Examples

The Four encoding examples.

① **INTEGER 14**

Tag identifies INTEGER, followed by length and encoded value.

② **OCTET STRING "HI"**

Tag identifies an octet string, followed by its length and bytes.

③ **OBJECT IDENTIFIER 1.3.6.1**

Encodes the hierarchical object identifier.

④ **IPAddress 131.21.14.8**

Encodes the four-byte IP address.

Key Point

BER provides a common encoding method so that systems with different architectures can exchange management information.

Example 9.1

Figure 9.9 shows how to define INTEGER 14. The size of the length field is from Table 9.1.

Figure 9.9 Example 9.1: INTEGER 14

0x02	0x04	0x00	0x00	0x00	0x0E
Tag (integer)	Length (4 bytes)	Value (14)			

Example 9.2

Figure 9.10 shows how to define the OCTET STRING "HL".

Figure 9.10 Example 9.2: OCTET STRING "HL"

0x04	0x02	0x48	0x49
Tag (String)	Length (2 bytes)	Value (H)	Value (L)

Example 9.3

Figure 9.11 shows how to define ObjectIdentifier 1.3.6.1 (iso.org.dod.internet).

Figure 9.11 Example 9.3: ObjectIdentifier 1.3.6.1

0x06	0x04	0x01	0x03	0x06	0x01
Tag (ObjectId)	Length (4 bytes)	Value (1)	Value (3)	Value (6)	Value (1)
----- 1.3.6.1 (iso.org.dod.internet) ----->					

Example 9.4

Figure 9.12 shows how to define IPAddress 131.21.14.8.

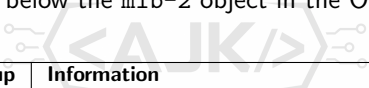
Figure 9.12 Example 9.4: IPAddress 131.21.14.8

0x40	0x04	0x83	0x15	0x0E	0x08
Tag (IPAddress)	Length (4 bytes)	Value (131)	Value (21)	Value (14)	Value (8)
----- 131.21.14.8 ----->					

9.2.5 MIB-2 – Management Information Base

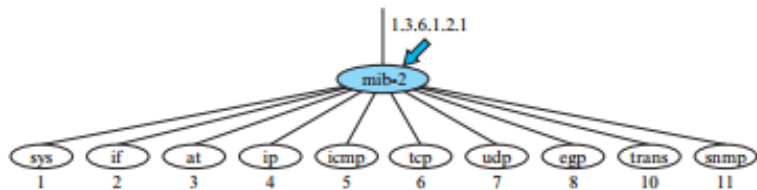
MIB-2 is the second component used in Internet network management.

- Each agent has its own MIB-2.
- It is a collection of objects that the manager can manage.
- Objects are organized into groups.
- Groups are located below the `mib-2` object in the OID tree.



Group	Information
sys	General system information
if	Network interface information
at	Address translation / ARP information
ip	IP information
icmp	ICMP information
tcp	TCP information
udp	UDP information
egp	EGP information
trans	Transmission information
snmp	SNMP information

Figure 9.13 Some mib-2 groups



Accessing MIB Variables – UDP Group

The **UDP group** provides an example of accessing MIB variables.

Variable	Object Identifier
udpInDatagrams	1.3.6.1.2.1.7.1
udpNoPorts	1.3.6.1.2.1.7.2
udpInErrors	1.3.6.1.2.1.7.3
udpOutDatagrams	1.3.6.1.2.1.7.4
udpTable	1.3.6.1.2.1.7.5

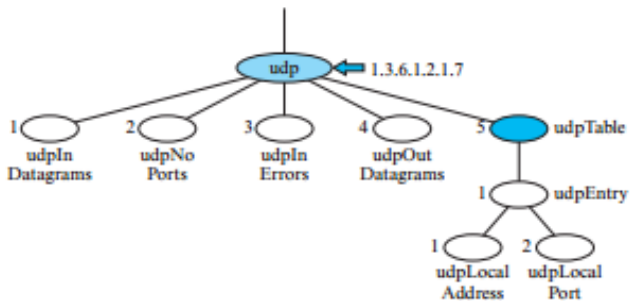
An OID identifies the **variable**, not its particular instance.
For a simple variable, append the instance suffix:

<code>udpInDatagrams.0 → 1.3.6.1.2.1.7.1.0</code>

Similarly:

`udpNoPorts.0 → 1.3.6.1.2.1.7.2.0`

Figure 9.14 *udp group*



MIB Tables and Indexing

A table is accessed through its hierarchical structure:

udpTable → udpEntry → Fields

udpTable	1.3.6.1.2.1.7.5
udpEntry	1.3.6.1.2.1.7.5.1
udpLocalAddress	1.3.6.1.2.1.7.5.1.1
udpLocalPort	1.3.6.1.2.1.7.5.1.2

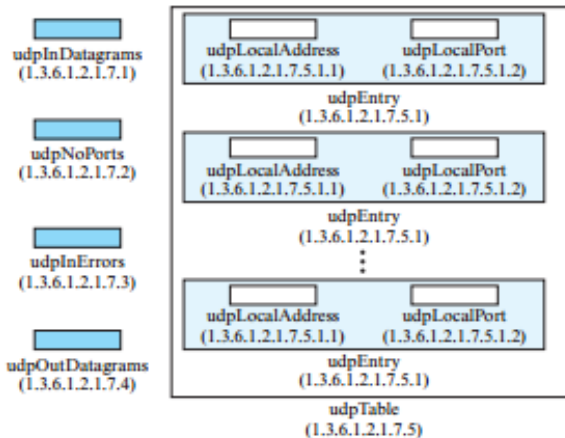
Important:

- A table itself is not a leaf object.
- The entry is also not a leaf.
- Individual fields are accessed at the leaf level.
- A table instance requires an index.
- The index may be based on one or more field values.

Example from the UDP table:

The index is based on the **local address and local port**.

Figure 9.15 *udp variables and tables*



MIB Table Index Example

A specific row of a table is identified by appending its **instance index** to the object identifier.

Example from the UDP table:

`udpLocalAddress.181.23.45.14.23`

The index contains:



Local Address + Local Port

For example:

`181.23.45.14 + 23`

Remember

Unlike ordinary programming arrays, MIB table indexes are based on the values of one or more fields in the table entry.

Figure 9.16 Indexes for udpTable

181.23.45.14	23
1.3.6.1.2.1.7.5.1.1.181.23.45.14.23	1.3.6.1.2.1.7.5.1.2.181.23.45.14.23
192.13.5.10	161
1.3.6.1.2.1.7.5.1.1.192.13.5.10.161	1.3.6.1.2.1.7.5.1.2.192.13.5.10.161
227.2.45.18	180
1.3.6.1.2.1.7.5.1.1.227.2.45.18.180	1.3.6.1.2.1.7.5.1.2.227.2.45.18.180
230.20.5.24	212
1.3.6.1.2.1.7.5.1.1.230.20.5.24.212	1.3.6.1.2.1.7.5.1.2.230.20.5.24.212

9.2.6 SNMP – PDUs

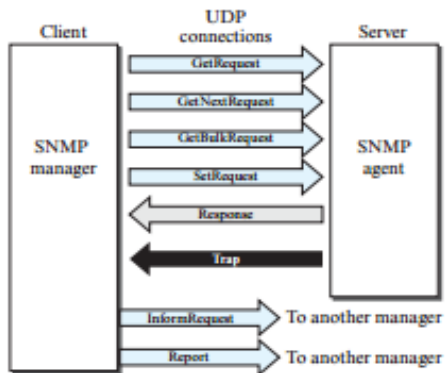
SNMP allows the manager to:

- Retrieve values from an agent.
- Store/change values in an agent.
- Receive alarm/event information from an agent.

SNMPv3 defines eight PDU types:

PDU	Tag	Purpose
GetRequest	A0	Retrieve variable value(s)
GetNextRequest	A1	Retrieve following object
Response	A2	Reply to request
SetRequest	A3	Store/change a value
GetBulkRequest	A5	Retrieve large amount of data
InformRequest	A6	Manager-to-manager request
Trap	A7	Agent reports an event
Report	A8	Reports certain errors

Figure 9.17 *SNMP PDUs*



SNMP PDU Format, Ports and Security

PDU Fields

- **PDU type**
- **Request ID** – matches request and response.
- **Error status**
- **Error index**
- **Variable-value pair list**
- **Non-repeaters** – GetBulkRequest.
- **Max-repetitions** – GetBulkRequest.

UDP Ports

161 – Agent

Requests and responses.

162 – Manager

Trap messages.

SNMPv3 Security

Provides configurable:

- Authentication
- Confidentiality
- Integrity

Also supports remote configuration of security aspects.

SNMP PDU Format, Ports and Security

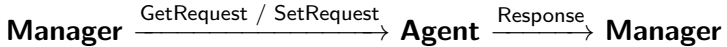
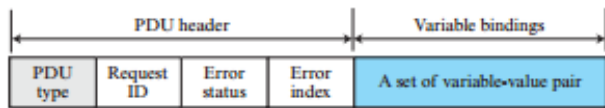


Figure 9.18 SNMP PDU format



a. All PDU types except GetBulkRequest

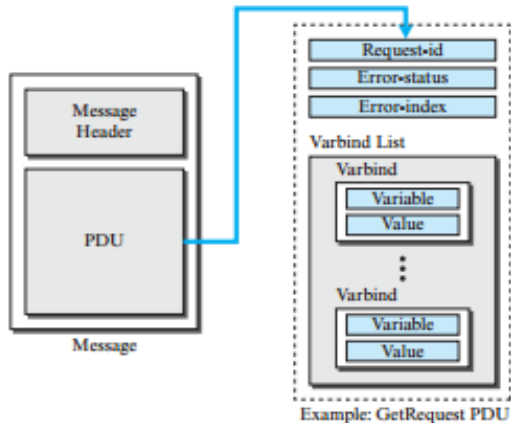


b. GetBulkRequest

Note:

The error status and error index values are set to 0 for all request messages.

Figure 9.19 *SNMP message*



Example 9.5 – SNMP GetRequest

Problem

A manager wants to retrieve the number of UDP datagrams received by a router. The corresponding MIB object is:

`udpInDatagrams.0`

with Object Identifier:

`1.3.6.1.2.1.7.1.0`

Manager constructs a GetRequest:

- PDU type: **GetRequest**
- Request ID identifies the request.
- Error status = **0** ,Error index = **0**
- One **VarBind** is included.
- Variable name: `1.3.6.1.2.1.7.1.0`
- Requested value is **NULL**.

Key Idea

In a **GetRequest**, the manager knows the **OID** of the variable but does not yet know its value.

Therefore:

OID → `udpInDatagrams.0`

Value → **NULL**

Figure 9.20 Example 9.5

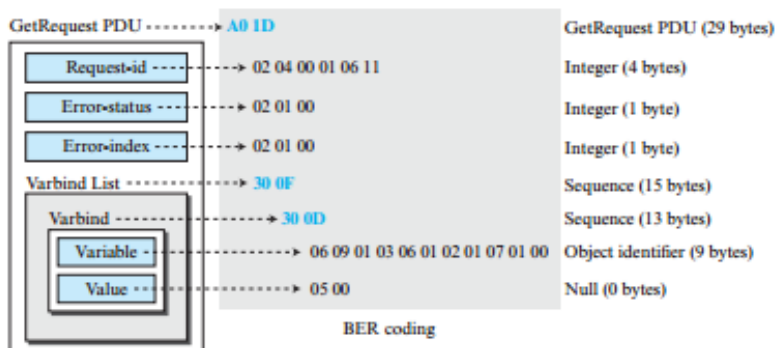


Figure 9.21 Actual message sent for Example 9.5

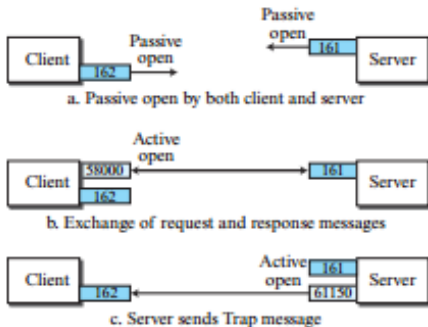
30	29	**	**
**	**	**	**
**	**	**	**
A0	1D	02	04
00	01	06	11
02	01	00	02
01	00	30	0F
30	0D	06	09
01	03	06	01
02	01	07	01
00	05	00	

Message

Note:

The byte values
are in hexadecimal

Figure 9.22 Port numbers for SNMP



Module 4: SNMP, ASN.1, Physical Layer



9.3 ASN.1 – Abstract Syntax Notation One

Why is ASN.1 Needed?

When a continuous stream of bits is transmitted, the receiver needs to know how the bits are organized and what they represent.

Simple Data

- Name + number
- Example:
 - First 12 bits → name
 - Next 8 bits → number

Complex Data

- Arrays
- Records
- Composite data types

Problem

The same number of bits can represent different structures.

Why Do We Need ASN.1?

Example: 2000-bit Message

A 2000-bit message could represent:

- An array of **200 numbers**, each of 10 bits, or
- An array of **10 numbers**, each of 200 bits.

Solution

Separate the **definition of data types** from the **sequence of bits transmitted**.

Data Type Definition



Encoded Bits

ASN.1

Abstract Syntax Notation One (ASN.1) is an abstract language that uses symbols, keywords, and atomic data types to create new data types.

Purpose in this Chapter

ASN.1 is a complex language used in different areas of computer science. Here, only the parts needed for understanding the **SNMP protocol** are introduced.

ASN.1 provides

- Symbols
- Keywords
- Primitive data types
- New data types
- Structured data types

Connection to SMI

The **Structure of Management Information (SMI)** uses a subset of ASN.1 entities in SNMP.

9.3.1 Language Basics – Symbols

ASN.1 uses single-character and multi-character symbols.

Symbol	Meaning	Symbol	Meaning
::=	Defined as / assignment	..	Range
	Or / alternative / option	{ }	Start and end of a list
-	Negative sign	[]	Start and end of tag
--	Comment	()	Start and end of subtype

Three Symbols to Remember

::= → Defined as

| → Choice

.. → Range

ASN.1 Symbols – Quick Examples

Assignment

Age ::= INTEGER

::= means “defined as”.

Choice

(sun | sat)

| represents an alternative.

Range

INTEGER (15..40)

.. defines a range.

Comment

-- This is a comment

ASN.1 Keywords

Keyword	Description
BEGIN	Start of a module
CHOICE	List of alternatives
DEFINITIONS	Definition of a data type/object
END	End of a module
EXPORTS	Data type exported to other modules
IDENTIFIER	Sequence identifying an object
IMPORTS	Data type imported from another module
INTEGER	Positive, zero, or negative integer
NULL	Null value
OBJECT	Used with IDENTIFIER

Keyword	Description
OCTET OF	Eight-bit binary data Used with SEQUENCE or SET
SEQUENCE	Ordered list
SEQUENCE OF	Ordered array of same type
SET	Unordered list
SET OF	Array of unordered lists
STRING	String of data

Convention

ASN.1 keywords should be written in **uppercase**.

9.3.2 ASN.1 Data Types

Basic Idea

ASN.1 provides several simple data types and allows them to be combined to create new simple or structured data types.

Simple / Atomic

Examples:

- INTEGER
- REAL
- BOOLEAN
- OCTET STRING
- OBJECT IDENTIFIER

Structured

- SEQUENCE
- SEQUENCE OF
- SET
- SET OF

Programming Analogy

Similar to C, C++, or Java:

Simple ASN.1 Built-in Types

Tag	Type	Set of Values
1	BOOLEAN	TRUE or FALSE
2	INTEGER	Positive, 0, or negative integers
3	BIT STRING	Binary digits or null set
4	OCTET STRING	String of octets or null set
5	NULL	Null, single valued
6	OBJECT IDENTIFIER	Values that define an object
7	ObjectDescriptor	Human-readable text describing an object
8	EXTERNAL	Type not in the standard
9	REAL	Real numbers in scientific notation
10	ENUMERATED	A list of integers
16	SEQUENCE / SEQUENCE OF	Ordered list of types
17	SET / SET OF	Unordered list of types
18	NumericString	Digits 0–9 and space
19	PrintableString	Printable characters
26	VisibleString	ISO646String
27	GeneralString	General character string
30	CHARACTER STRING	Character set

New Data Types

ASN.1 uses **Backus–Naur Form (BNF)** syntax for defining a new data type.

General Syntax

`<new type> ::= <type>`

- A new type can be created from a built-in data type.
- A new type can also be based on a previously defined type.
- The new type must start with a **capital letter**.

Simple Examples

`Married ::= BOOLEAN` `Age ::= INTEGER`

Example 9.6 – New Data Types

Definitions

`Married ::= BOOLEAN`

`MaritalStatus ::= ENUMERATED`
`{single, married, widowed, divorced}`

`DayOfWeek ::= ENUMERATED`
`{sun, mon, tue, wed, thu, fri, sat}`

`Age ::= INTEGER`

Married

- New type based on BOOLEAN.

Age

- New type based on INTEGER.

ENUMERATED

- Defines a list of possible choices.
- `DayOfWeek` contains seven choices.

New Subtypes

ASN.1 allows us to create a subtype whose range is a **subrange** of a built-in or previously defined data type.

Range

The symbol (..) defines a range.

```
INTEGER (15..40)
```

Only values from 15 through 40 are permitted.

Choice

The symbol | defines a choice.

```
(sun | sat)
```

The value can be either sun or sat.

Example 9.7 – New Subtypes

Subtype Definitions

`NumberOfStudents ::= INTEGER (15..40)`

`Grade ::= REAL (1.0..5.0)`

`Weekend ::= DayOfWeek (sun | sat)`

NumberOfStudents

Grade

$15 \leq \text{NumberOfStudents} \leq 40$

$1.0 \leq \text{Grade} \leq 5.0$

Weekend

The value must be either `sun` or `sat`.

Simple Variables

Value Name

In ASN.1, the term **Value Name** is used instead of the term variable, although the concept is familiar to programmers.

Syntax

```
<variable> <type> ::= <value>
```

- A variable has a particular type.
- The assigned value must belong to the range of that type.
- The variable name starts with a **lower-case letter**.

Example

```
numberOfComputers INTEGER ::= 2
```

Example 9.8 – Variables and Values

Examples

```
numberOfComputers INTEGER ::= 2
married Married ::= FALSE
herAge INTEGER ::= 35
classSize NumberOfStudents ::= 22
```

Built-in types

- numberOfComputers
- herAge

Defined types

- married uses Married
- classSize uses
NumberOfStudents

Structured Type – SEQUENCE

SEQUENCE

ASN.1 uses the keyword **SEQUENCE** to define a structured data type similar to a **struct/record** in C or C++.

StudentAccount

```
StudentAccount ::= SEQUENCE
{
    userName VisibleString,
    password VisibleString,
    accountNumber INTEGER
}
```

- SEQUENCE is an **ordered list** of variable types.
- Components can have different data types.
- Here, there are three components.

Structure Variables

After defining a structured type, we can create a variable of that type and assign values.

Example

```
johnNewton StudentAccount
{
    userName "JohnN",
    password "120007",
    accountNumber 25579
}
```

Structure

- userName
- password
- accountNumber

Assigned Values

- "JohnN"
- "120007"
- 25579

Figure 9.23 – Structure Variable

Figure 9.23 – Record Created from Type Definition and Value Assignments

What the Figure Shows

The record created from the `StudentAccount` type definition after values have been assigned to its components.

$$\text{johnNewton} \rightarrow \begin{cases} \text{userName} = \text{"JohnN"} \\ \text{password} = \text{"120007"} \\ \text{accountNumber} = 25579 \end{cases}$$

Structure Variables – Example 9.8

Structure Variable

After defining the `StudentAccount` type, a variable can be created and values can be assigned to its components.

Figure 9.23 *Record representing the type definition and variable declaration*

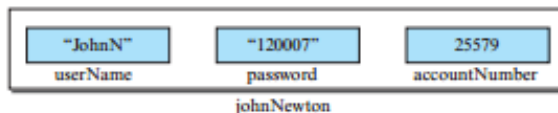


Figure 9.23

Record created from the type definition and value assignments.

SEQUENCE OF – Array-like Structure

Definition

The keyword **SEQUENCE OF** defines a composite type in which all components are of the **same type**.

Similar to an Array

- Components have the same type.
- Components are ordered.
- Similar to an array in C/C++.

Router Forwarding Table

A forwarding table can be defined as:

SEQUENCE OF Rows

Each Row is itself a SEQUENCE containing several variables.

Remember

SEQUENCE → Ordered list of variable types

SEQUENCE OF → Ordered collection of same type

9.3.3 ASN.1 – Encoding

Encoding

After data has been defined and values have been associated with variables, ASN.1 uses one of the encoding rules to encode the message for transmission.

1. Define

Data types

2. Assign

Values

3. Encode

Message

Connection with Previous Section

The **Basic Encoding Rule (BER)** was discussed in the SNMP section. ASN.1 defines the data structure, while encoding rules define how it is represented for transmission.

Key Points

- ASN.1 separates data definition from transmitted bits.
- It provides symbols, keywords, and built-in data types.
- New types and subtypes can be defined.
- **SEQUENCE** is similar to a record/struct.
- **SEQUENCE OF** is similar to an array.
- Encoding rules convert the defined data into a form suitable for transmission.